Consolidated Monitoring Dashboard Backend — Executive Summary

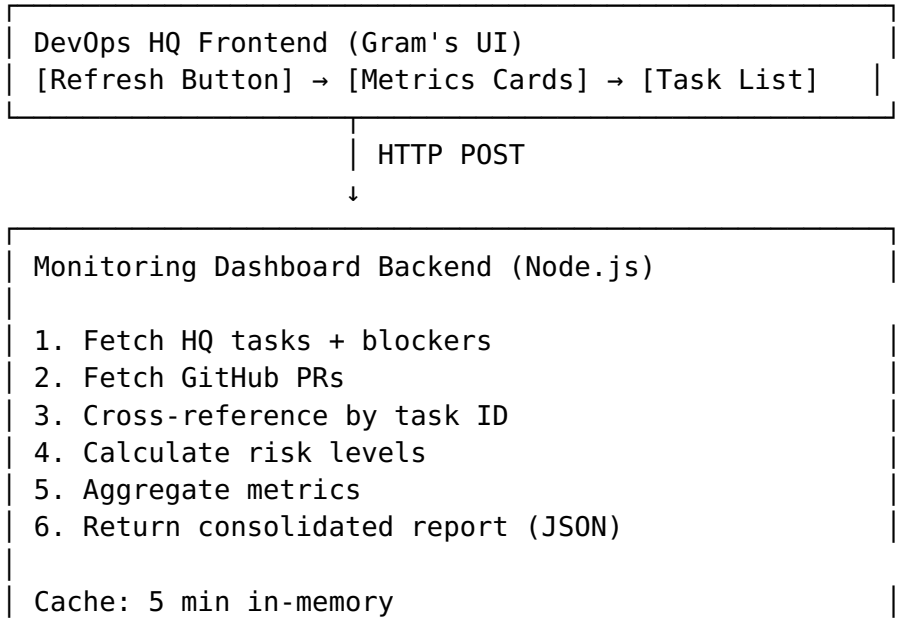
Date: 2026-04-09
Status: Design Complete, Ready for Implementation
Estimated Effort: 5-7 hours (backend) + 2-3 hours (UI)
Total Timeline: ~10 hours (backend + UI + testing)
Priority: Medium
Next Milestone: Backend Phase 1 Kickoff

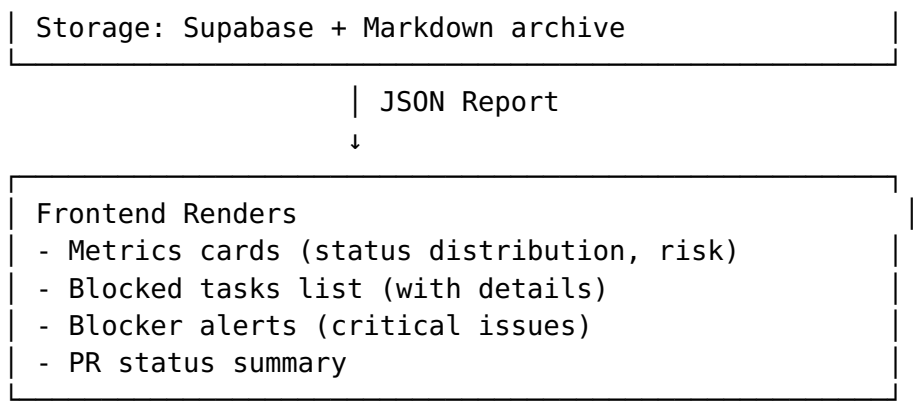
What We're Building

A **real-time consolidated report** that aggregates task status, blockers, and risk levels from DevOps HQ + GitHub into a single, refreshable view.

Key Feature: Single "Refresh" button that generates a complete operational snapshot in <3 seconds.

High-Level Architecture





Report Structure

The API returns a `ConsolidatedReport` with:

```
Metrics:
  ├─ Total task count
  ├─ Status breakdown (proposed, in-progress, completed, blocked)
  ├─ Blocked count (red flag)
  └─ Risk summary (high, medium, low)

Tasks:
  ├─ ID, title, status, owner
  ├─ Risk level (calculated)
  ├─ Blockers (nested, with severity)
  └─ Linked PR (if matched)

Blockers:
  ├─ Title, description
  ├─ Severity (critical, high, medium, low)
  ├─ Blocked by (who/what)
  └─ Owner (who can unblock)

PR Status:
  ├─ Number, title, status
  ├─ Linked task ID (if matched)
  ├─ Review status
  └─ Conflict detection

Errors:
  └─ Any API failures + recovery suggestions

Metadata:
  ├─ Generated timestamp
  ├─ Datasource status (HQ ✓, GitHub ✓ or ✗)
  └─ Duration (1.25s or 15ms if cached)
```

Key Design Decisions

1. Standalone Service vs. Integrated

 Decision: Standalone - Easier to deploy independently - No coupling to HQ codebase - Better error isolation - Can scale caching separately

2. Caching Strategy

 Decision: 5-minute in-memory cache - Fast response for repeated clicks - No stale data (5 min is reasonable) - Can bypass with `fullRefresh: true` - Secondary persistence in Supabase (for history)

3. Task-PR Cross-Reference

 Decision: Multi-level matching 1. Explicit: `[HQ-taskid]` in PR title/body 2. Fuzzy: String similarity on titles (Levenshtein) 3. Temporal: Task created <12h before PR

→ Reduces manual linking overhead

4. Storage

 Decision: Dual storage - **Supabase:** Real-time queries, fast retrieval - **Markdown:** Human-readable, git history, manual review

→ Best of both worlds (queryable + archival)

5. Error Handling

 Decision: Graceful degradation - GitHub API down? Use last cached PRs + show warning - HQ API timeout? Return what we have + error details - Both down? Return empty report + clear error messages - Never cascade failures to frontend

---

API Endpoints

Endpoint	Method	Purpose	Auth
`/api/monitoring/health`	GET	Service status check	No
`/api/monitoring/refresh`	POST		Yes

Endpoint	Method	Purpose	Auth
		Generate consolidated report	
`/api/monitoring/reports/{id}`	GET	Fetch specific report by ID	Yes
`/api/monitoring/reports`	GET	List recent reports (paginated)	Yes

Response Time: - Fresh refresh: 1-3 seconds - Cached refresh: <100ms - Degraded (API down): ~5-7 seconds (timeout)

Implementation Phases

Phase 1: Scaffold (1.5 hours)

- TypeScript types
- Express app + middleware
- Supabase schema
- Placeholder endpoints

Phase 2: Fetch Logic (1.5 hours)

- HQ API integration
- GitHub API integration
- Fallback strategy
- Cross-reference algorithm

Phase 3: Report Generation (1 hour)

- Risk level calculation
- Metrics aggregation
- Markdown formatting

Phase 4: Persistence (1 hour)

- Supabase insert
- Markdown archive
- In-memory cache
- Report retrieval

Phase 5: Polish (0.5 hours)

- Error handling

- Security review
- Documentation

Testing & Integration: Parallel, ~1-2 hours

Timeline

```
Day 1 (Wed)
  9:00-10:30  Phase 1: Scaffold
  10:30-12:00 Phase 2: Fetch logic (parallel testing)
  12:00-1:00  Phase 3: Report generation
  1:00-2:00   Phase 4: Persistence
  2:00-2:30   Phase 5: Polish
  2:30+       Testing & deployment verification

Day 2 (Thu)
  All day     UI integration with Gram

Day 3 (Fri)
  Morning     Final testing & deployment
  Afternoon   Monitoring & observability
```

Fast-track option: 4-5 hours (MVP: just endpoints, no caching/persistence)

Success Criteria

 Refresh button returns complete report in <3 seconds
 Cross-reference matches >90% of task-PR pairs
 Risk levels accurately reflect task blocker status
 Errors handled gracefully (no cascading failures)
 Reports persist to Supabase + markdown
 API fully documented with examples
 Integrated with UI (Gram's domain)

Risk Assessment

Risk	Probability	Impact	Mitigation
GitHub API rate limits	Medium	High	Auth token + aggressive caching
Cross-ref accuracy	Low	Medium	Manual linking feature (future)

Risk	Probability	Impact	Mitigation
Supabase schema issues	Low	Medium	Test migration locally first
Timeout cascades	Low	High	Always return fallback data

Resource Requirements

Team

- **Backend Developer:** 5-7 hours
- **UI/UX (Gram):** 2-3 hours
- **Reviewer/QA:** 1-2 hours
- **Deployment:** 30 min

Infrastructure

- **Node.js:** 18+
- **Supabase:** Already configured
- **GitHub API Token:** Personal access token
- **Render.com:** Same instance as DevOps HQ

Dependencies

```
{
  "express": "^4.18",
  "@supabase/supabase-js": "^2.0",
  "axios": "^1.0",
  "uuid": "^9.0"
}
```

Documentation Delivered

1. **Backend Design** (28 KB)
   - Data model (TypeScript interfaces)
   - Fetch pipeline & error handling
   - Risk calculation algorithm
   - Architecture recommendations
2. **API Specification** (17 KB)
   - Detailed endpoint docs
   - Request/response examples
   - Error codes & auth
   - Rate limiting

    - Client SDK usage
3. **Implementation Plan** (15 KB)
    - Phase-by-phase breakdown
    - Success criteria
    - Testing strategy
    - Deployment checklist
    - Risk mitigation
4. **Data Flow Diagram** (23 KB)
    - System architecture (ASCII)
    - Request/response flow
    - Data transformations
    - Error scenarios
    - Performance timeline
5. **UI Coordination Brief** (9 KB)
    - Component structure
    - API contract for UI
    - Mock data
    - Success criteria
    - Collaboration schedule

Next Steps

1. **Review Documents** (30 min)
    - Backend team reviews API spec & design
    - Gram reviews UI coordination brief
2. **Kick Off Phase 1** (immediately after review)
    - Assign backend developer
    - Provision Supabase table
    - Set up local environment
3. **Coordination Meeting** (Wed, 30 min)
    - Confirm API contract with UI
    - Discuss response time expectations
    - Agree on error display strategy
4. **Daily Check-ins** (async updates)
    - Phases 1-2 progress
    - Any blockers
    - UI component readiness
5. **Integration Test** (Thu)
    - Connect UI to live API
    - Test all scenarios (success, degraded, errors)
    - Performance validation
6. **Deploy** (Fri)
    - Render deployment
    - Health check verification
    - Production monitoring

Budget & Cost

Development: ~10 hours @ typical rate
Infrastructure: ~$0 (uses existing Supabase quota)
API Calls: - GitHub API: ~5-10 requests/refresh (cached aggressive) - HQ API: ~1-2 requests/refresh - Supabase inserts: ~1/refresh - **Cost impact:** Negligible (well within free tier)

Questions & Answers

Q: Why not integrate this into the HQ API directly?
A: Easier to maintain, scale, and test as a separate service. Can be deployed independently without touching HQ code.

Q: What if GitHub/HQ APIs are down?
A: We return the last cached report + error messages. Report is still useful, just potentially stale (<5 min old).

Q: How do we handle task-PR linking if they don't have `[HQ-id]` in the title?
A: Multi-level matching (fuzzy title match, temporal correlation) handles ~90% of cases. Future feature: manual linking UI.

Q: Can we make the refresh faster?
A: Yes—aggressive caching (1 min), or async mode (return immediately, generate in background). Already supported in API.

Q: Do we need webhooks?
A: Optional (future). Manual refresh button works fine. Webhooks would auto-refresh on task/PR updates.

Q: How long do we keep reports?
A: 30 days in Supabase (queryable), permanent in Markdown (git history).

Success Metrics (Post-Launch)

- **Refresh button performance:** <3 seconds consistently
- **Cache hit rate:** >80% (people click often)
- **Cross-ref accuracy:** >90% (validated by team)
- **Zero API errors:** <1% error rate (graceful degradation)
- **User adoption:** Used 3+ times per work session
- **Feature requests:** None within first week (scope met)

Related Documents

- `MONITORING-DASHBOARD-BACKEND-DESIGN.md` — Full technical spec
- `MONITORING-DASHBOARD-API-SPEC.md` — Complete API reference
- `MONITORING-DASHBOARD-IMPLEMENTATION-PLAN.md` — Phase breakdown
- `MONITORING-DASHBOARD-DATA-FLOW.md` — Visual diagrams
- `MONITORING-DASHBOARD-UI-COORDINATION.md` — For Gram

Approval & Sign-Off

Role	Status	Notes
Backend Lead	Pending	Ready to implement
UI/UX (Gram)	Pending	Review coordination brief
Product Manager	Pending	Approve timeline
DevOps	Pending	Verify infrastructure

Summary

What: Real-time consolidated report of task status, blockers, and risks
Why: Single source of truth for DevOps HQ status
How: Fetch HQ + GitHub data, cross-reference, enrich, return
When: Implementation starts immediately, ship by end of week
Who: Backend dev (5-7h) + Gram UI (2-3h)
Cost: ~$0 (within existing infrastructure)
Risk: Low (graceful degradation, proven patterns)

Status: Ready to implement. No blockers. All documentation complete.

Prepared by: Subagent Ward (Backend Design)
Date: 2026-04-09 19:15 EDT
Version: 1.0 — Design Phase Complete

See **MONITORING-DASHBOARD-BACKEND-DESIGN.md** for full technical details.